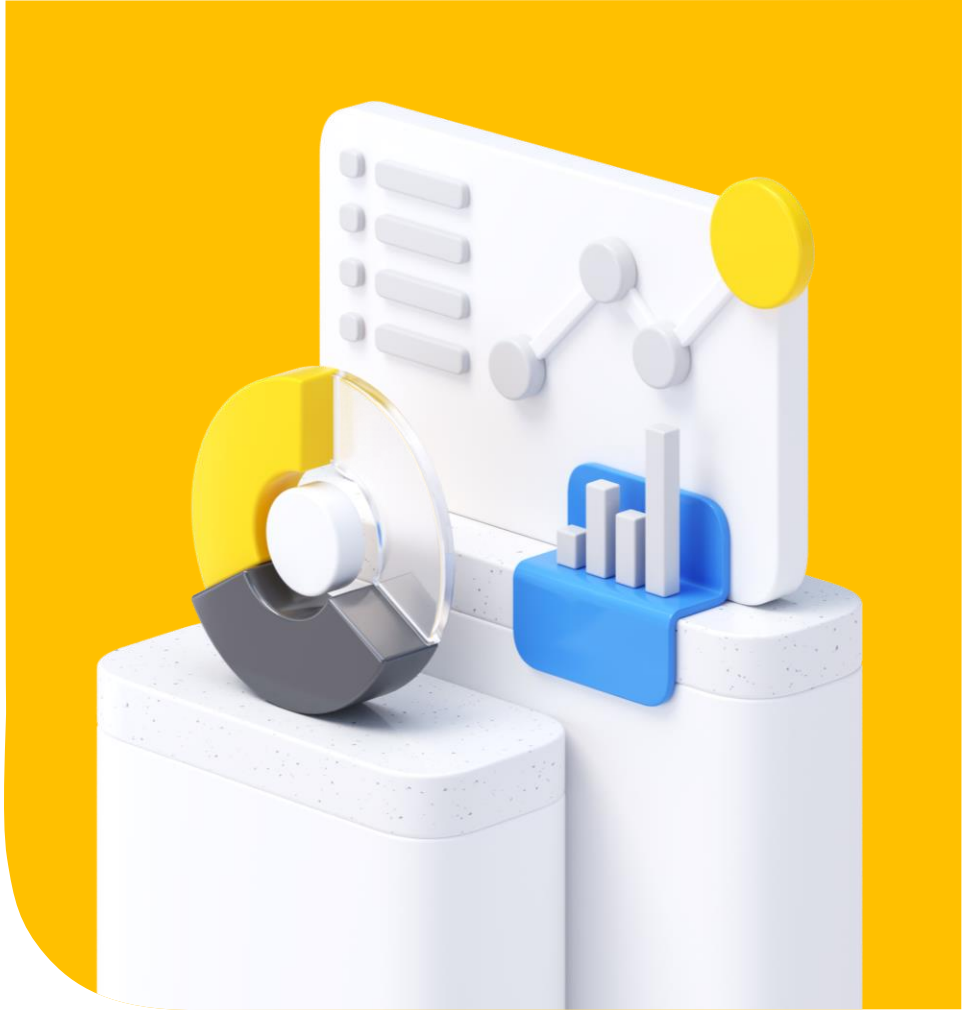


Лекция 10

Хранение данных

Обо мне

План урока

Варианты хранения

Core Data

Альтернативы

Варианты хранения

Зачем вообще?

Что мы храним на девайсе

- Профиль пользователя
- Пароли и токены
- Настройки приложения
- Кэш сетевых запросов
- Бизнес модели
- Картинки и прочие ресурсы

UserDefaults

Используется для хранения пользовательских настроек

- Bool, Int, Float, Double, Dictionary, Array, String, Data
- Не подходит для чувствительных данных
- Шаринг данных между процессами (виджеты, часы ...)

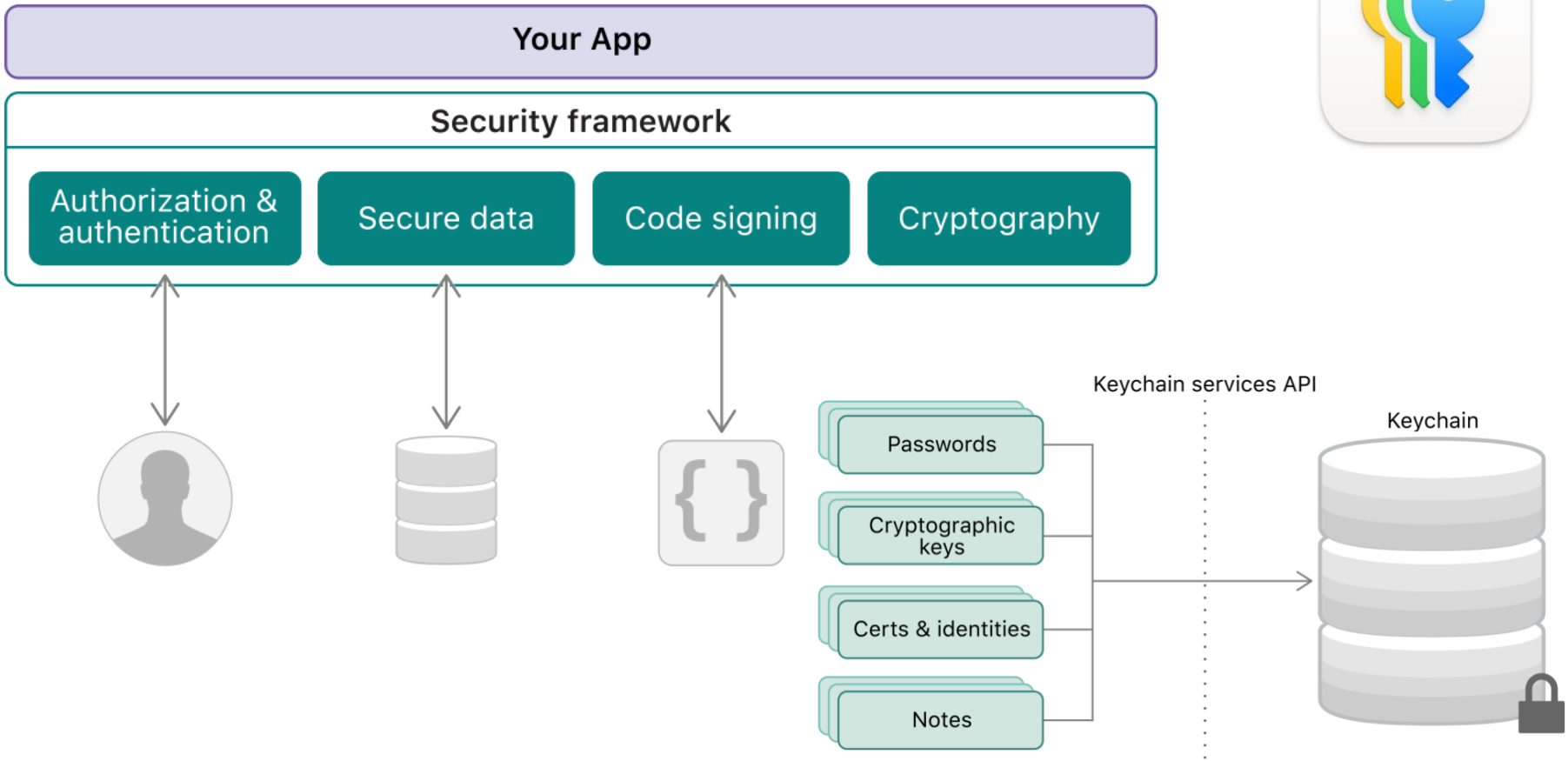
FileManager

- Сохранение и загрузка пользовательских файлов
- Музыка, изображения, текстовые файлы, видео ...
- Отдельная песочница

Sensitive data

- Логины, пароли и токены (JWT)
- Ключи шифрования (AES, RSA)
- SSL сертификаты
- Конфиденциальная информация: (номер кредитной карты, адреса)

Security (Keychain services)



Keychain services

Основные методы:

- SecItemAdd — добавление значения
- SecItemCopyMatching — чтение
  SecItemUpdate — обновление
- SecItemDelete — удаление

Итоги

В этом разделе мы узнали:

- Что и где храним
- UserDefaults
- FileManager
- Keychain services

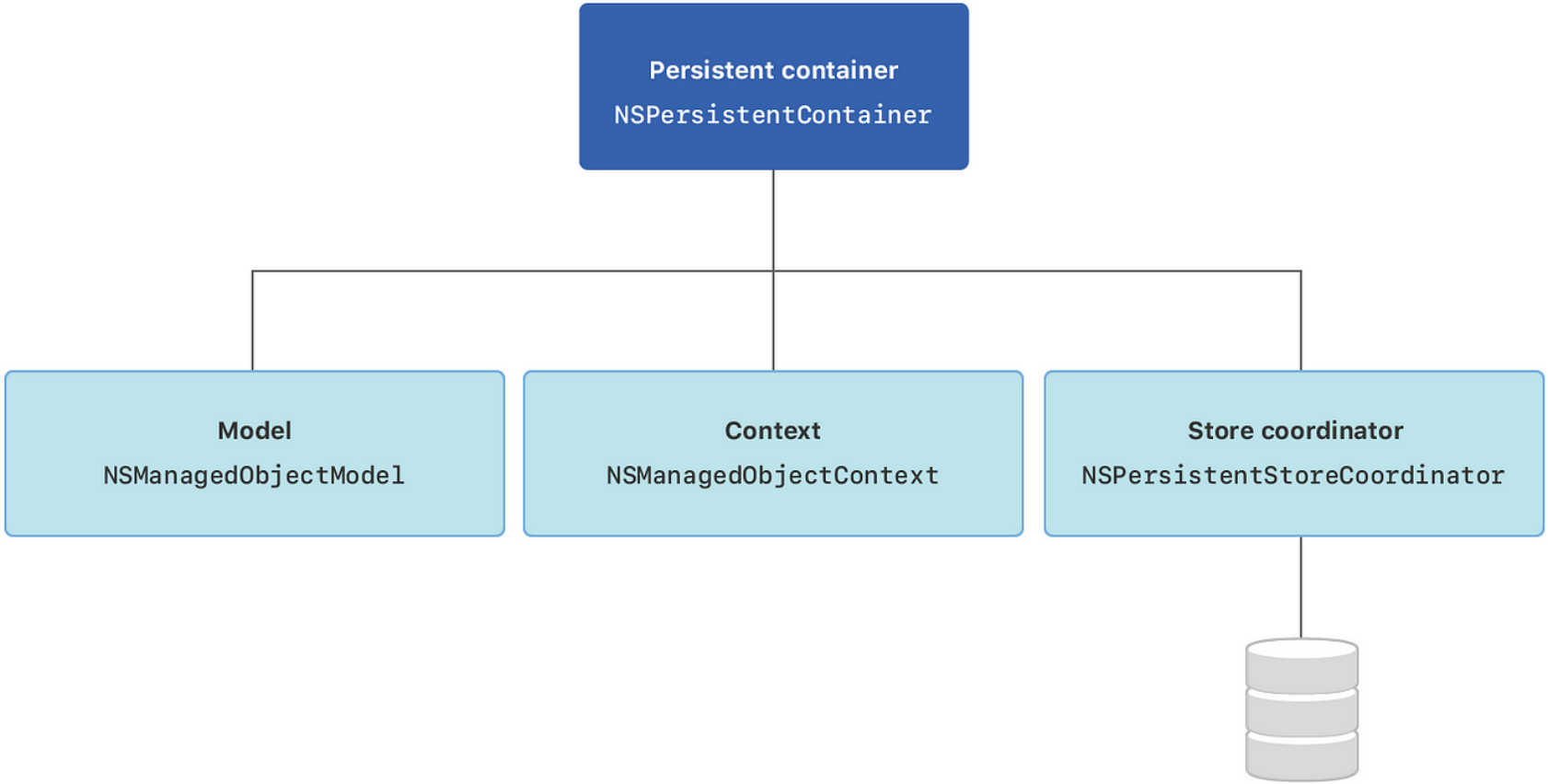
Вопросы?

CoreData

CoreData

- Разработан и поддерживается Apple
- Нативный фреймворк для iOS и Mac OS
- Декларативный подход для работы с базой данных
- Создание, добавление, удаление моделей данных
- Построение связей между моделями
- Поиск моделей по условиям

CoreData


Persistent container
NSPersistentContainer
Model
NSManagedObjectModel
Context
NSManagedObjectContext
Store coordinator
NSPersistentStoreCoordinator

NSPersistentStore

Абстрактный класс, представляющий из себя постоянное хранилище

- Типы хранилища:
- SQLite - создается и используется в CoreData по дефолту
- XML - только для Mac OS
- In-Memory - используется для кеша
- Custom - нужно реализовать сабкласс от NSAtomicStore/ NSIncrementalStore

NSManagedObjectModel

Объект, который описывает схему - коллекцию сущностей (моделей данных), которые используются в приложении.

NSPersistentStoreCoordinator

Класс, предоставляющий доступ к управлению хранилищем

- Создает хранилище на основе модели-схемы
- Конфигурирует свойства хранилища
- Осуществляет миграцию данных между моделями
- Запись в Store блокирует чтение данных

NSPersistentContainer

Доработанный NSPersistentStoreCoordinator, доступен начиная с iOS 10

- Скрывает сложность по созданию стека Core Data
- Предоставляет готовые ViewContext и BackgroundContext
- Параллельно одна запись и множество чтений из Store

NSManagedObjectContext

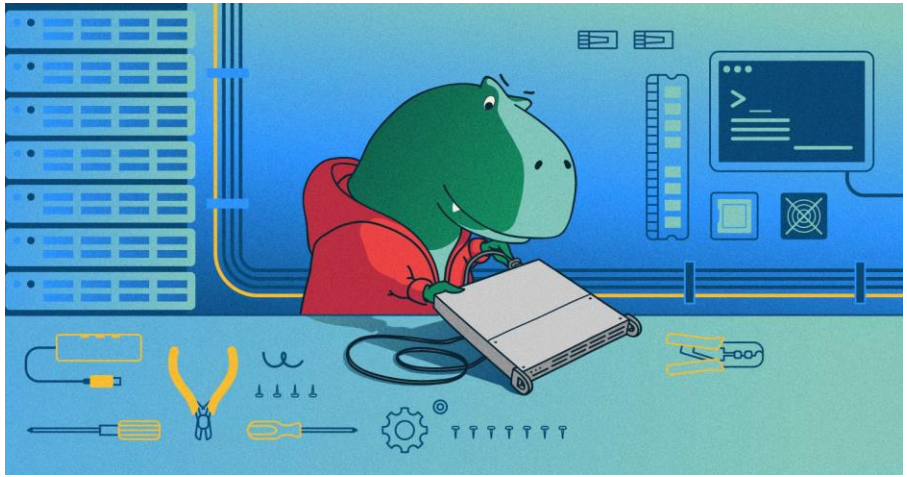
Сущность - среда для работы с объектами Core Data

- Сохраняет, извлекает, удаляет объекты
- Позволяет следить за изменениями объектов
- Менеджер транзакций save() и rollback()
- Main Context - для чтения изменений и вывода в UI
- Background Context позволяет асинхронно обрабатывать массивы данных
- Не используй Main Queue для записи сущностей!

NSManagedObject

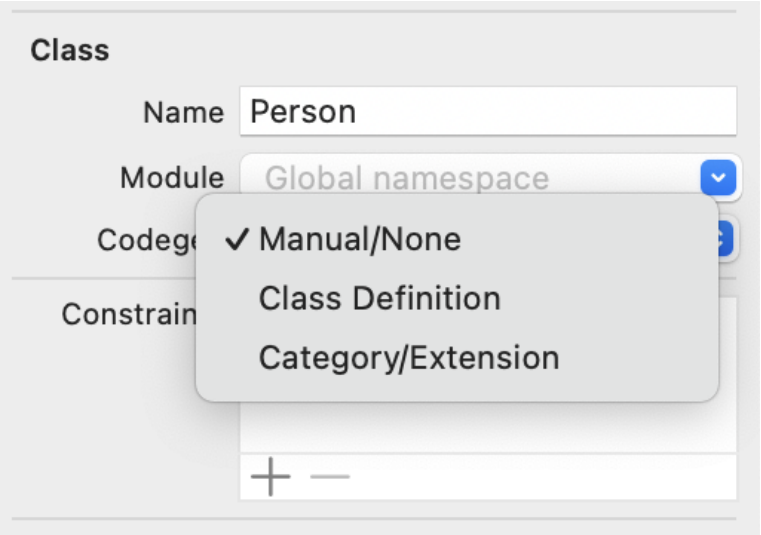
Управляемый объект, экземпляр сущности

- Аналог записи в таблице в классической БД
- Класс генерируется автоматически из модели
- Для каждого context используй свой инстанс NSManagedObject

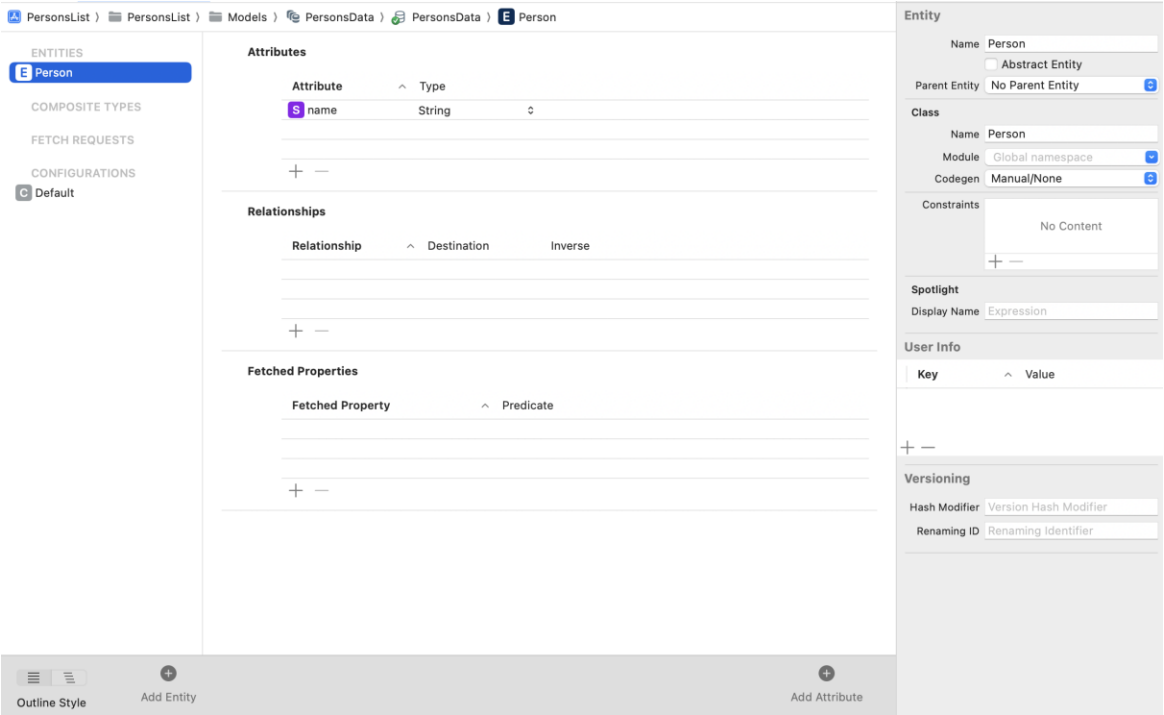


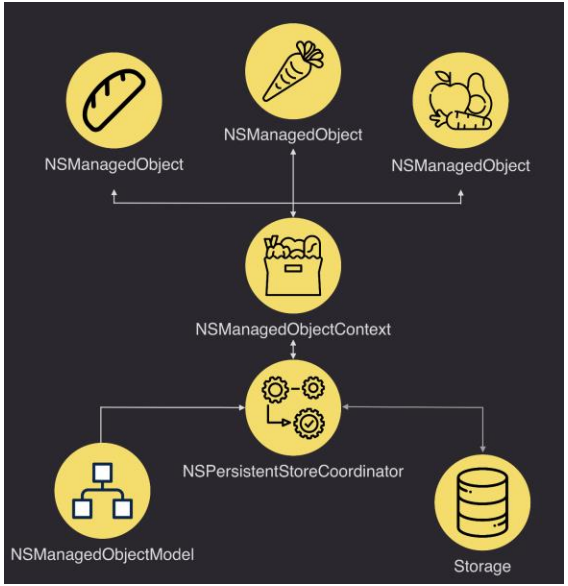
Кодогенерация NSManagedObject

- Полностью вручную
- Автоматическое создание класса + extensions (по умолчанию)
- Автоматическое создание extensions



Модель данных CoreData





Итоги

В этом разделе мы узнали:

- Что такое Core Data
- Из каких компонент состоит Core Data
- Как организовать компоненты
- Подходы к сохранению и записи данных

Вопросы?

Альтернативы

Realm

```
class Dog: Object {
    @Persisted var name: String
}
```

```
do {
    let realm = try Realm()
    try realm.write() {
        let dog = Dog()
        dog.name = "Alex"
        realm.add(dog)
    }
} catch {
    print("Error")
}
```

```
let dogs = realm.objects(Dog.self)
```

```
realm.delete(dog)

realm.delete(realm.objects(Dog.self))
```

Realm

Docs Home / MongoDB Atlas / Atlas Device SDKs

Atlas Device SDKs Deprecation

Atlas Device SDKs are deprecated. Refer to the **deprecation page** for details.

As of September 2024, Atlas Device SDKs are deprecated. Atlas Device SDKs will reach end-of-life and be removed on **September 30, 2025**.

The Atlas Device SDKs are made up of two parts: the on-device database and Atlas Device Sync. The on-device database will continue to exist as an open source project.

Atlas Device Sync is deprecated. This means that apps built using Sync need to move to an alternative solution or remove Sync before September 30, 2025. Refer to the App Services deprecation page for more information.

Warning

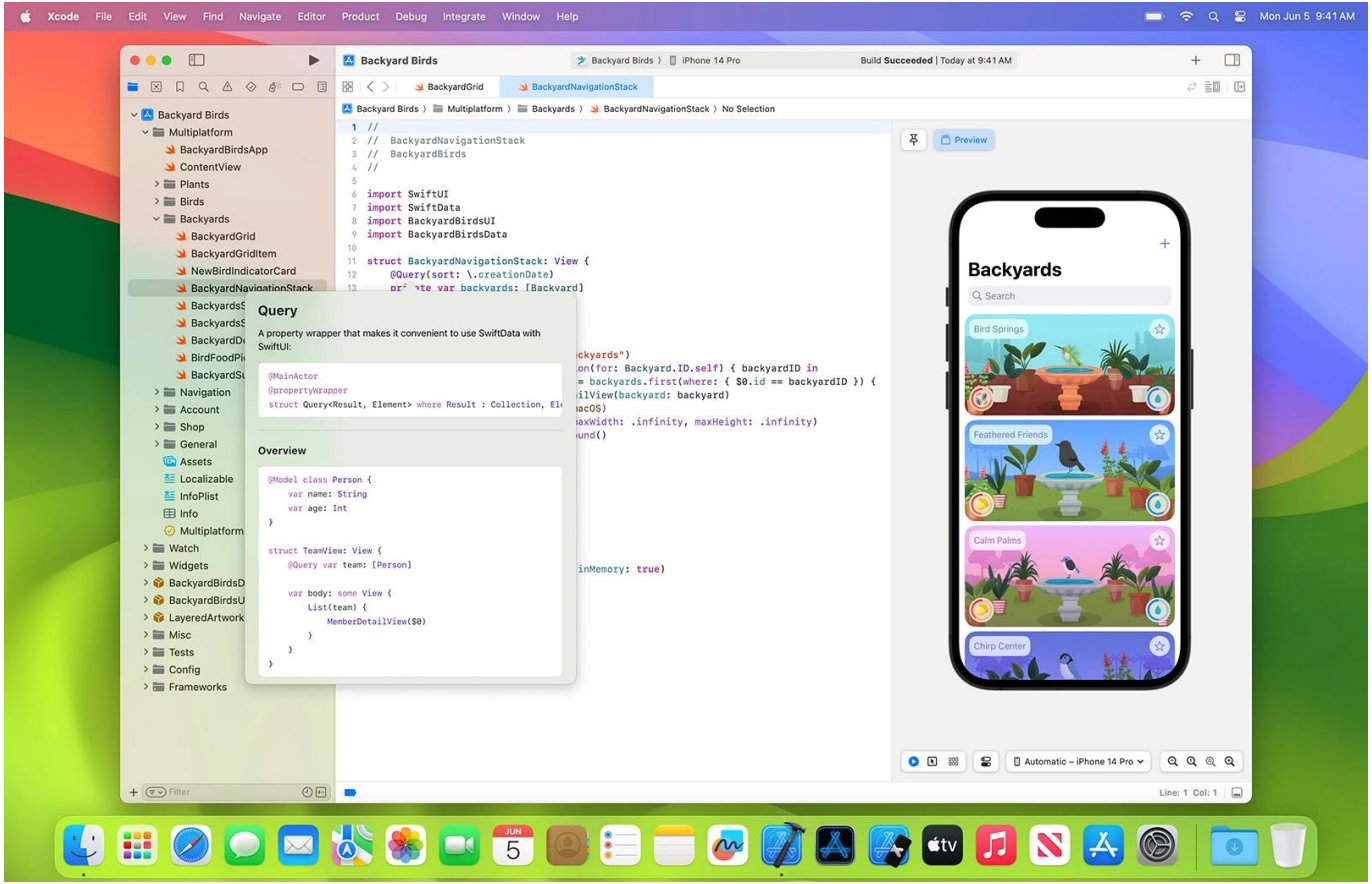
We announced the deprecation of Atlas Device Sync + Realm SDKs in September 2024. For more information please see:

- SDK Deprecation
- Device Sync Deprecation

For a version of `realm-swift` without sync features, install version 20 or see the `community` branch.

SwiftData

WWDC24

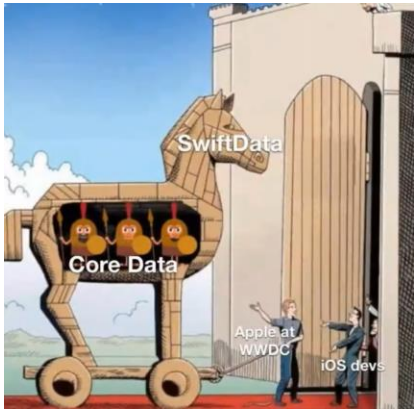

Xcode File Edit View Find Navigate Editor Product Debug Integrate Window Help
Mon Jun 5 9:41 AM
Backyard Birds
Backyard Birds 〉 iPhone 14 Pro
Build Succeeded | Today at 9:41 AM
BackyardGrid
BackyardNavigationStack
Backyard Birds 〉 Multiplatform 〉 Backyards 〉 BackyardNavigationStack 〉 No Selection
Backyard Birds
Multiplatform
BackyardBirdsApp
ContentView
Plants
Birds
Backyards
BackyardGrid
BackyardGridItem
NewBirdIndicatorCard
BackyardNavigationStack
Navigation
Account
Shop
General
Assets
Localizable
InfoPlist
Info
Multiplatform
Watch
Widgets
Misc
Tests
Config
Frameworks
//
// BackyardNavigationStack
// BackyardBirds
//
import SwiftUI
import SwiftData
import BackyardBirdsUI
import BackyardBirdsData
struct BackyardNavigationStack: View {
@Query(sort: \.creationDate)
Query
A property wrapper that makes it convenient to use SwiftData with SwiftUI:
@MainActor
@propertyWrapper
struct Query<Result, Element> where Result : Collection,
Overview
@Model class Person {
var name: String
var age: Int
}
struct TeamView: View {
@Query var team: [Person]
var body: some View {
List(team) {
MemberDetailView($0)
}
}
}
Preview
Backyards
Search
Bird Springs
Feathered Friends
Calm Palms
Chirp Center
Automatic – iPhone 14 Pro
Line: 1 Col: 1

SwiftData

```swift
@Model class Trip {
    @Attribute(.preserveValueOnDeletion)
    var name: String

    @Relationship(deleteRule: .cascade, inverse: \LivingAccommodation.trip)
    var livingAccommodation: LivingAccommodation?

    init(name: String) {
        self.name = name
    }
}
```




```swift
let modelID = change.changedPersistentIdentifier
let fetchDescriptor = FetchDescriptor<Trip>(predicate: #Predicate {
    $0.livingAccommodation?.persistentModelID == modelID
})
if let matchedTrip = try? taskContext.fetch(fetchDescriptor).first {
    switch change {
    case .insert:
        resultTrips.insert(matchedTrip)
    case .update:
        resultTrips.update(with: matchedTrip)
    case .delete:
        resultTrips.remove(matchedTrip)
    default:
        break
    }
}
```

Спасибо

Хакатон